

COLLECTION TYPES

Collections (or) Data Structures (or) Collection Types:

A collection is an object that represents group of objects(elements).

A data structure defines a particular way of storing and organizing data in a system so that it can be used efficiently.

Advantages of Collection Types:

- 1) Reduces programming effort
- 2) Increases programming speed and quality

There are five collection types in python:

1) list 2) tuple 3) set 4) frozenset 5) dict

All collection types are classes in python.

Collection objects are iterable objects.

Every collection class contains methods that are used to perform operations on elements.

List:

1) List can be created by using square brackets.

```
a=[];  
print(a);  
print(type(a))
```

2) List can also be created by using list() function.

```
a=list();  
print(a);  
print(type(a))
```

3) List can have homogeneous elements and heterogeneous elements.

```
1) a=[10,20,15,25,30];  
   print(a);  
   print(type(a))  
2) a=[10, 10.2, "abc", True];  
   print(a);  
   print(type(a))
```

4) List allows duplicates.

```
a=[10,20,10,50,88,31,89,88]
```

5) List supports both positive and negative indexing.

```
a=[10,20,10,50,88,31,89,88];  
print(a);  
print(a[1]);  
print(a[-3])
```

6) List supports slice operations also.

```
a=[10,20,10,50,88,31,89,88];  
print(a);  
print(a[1:3]);  
print(a[-4:-1])
```

7) List is mutable.

```
a=[10,20,30,13,56,93];  
print(a);  
print(id(a));  
a[1]=22;
```

```
print(a);  
print(id(a))
```

8) List supports deleting an element.

```
a=[10,20,30,13,56,93];  
print(a);  
del a[2];  
print(a)
```

9) List is an iterable object, elements of a list can be iterated by using for loop or while loop.

```
1) a=[10,33,63,98,13,67]
```

```
for b in a:
```

```
    print(b)
```

```
2) a=[10, 98, 32, 81, 38, 90]
```

```
i=0
```

```
while i<len(a):
```

```
    print(a[i])
```

```
    i+=1
```

10) Insertion order is preserved in a list.

```
1) a=[10, 20, 30] => list
```

```
print(a) => insertion order preserved
```

```
2) a={11, 53, 33} => set
```

```
print(a) => insertion order not preserved
```

11) List can be placed in another list also. List in a list called as nested list.

```
a=[[10,11,12],[20,21,22],[30,31,32]]
```

```
for b in a:
```

```
print(b)
for c in b:
    print(c)
```

12) List allows search operation also.

```
a=[10, 98, 32, 81, 38, 90]
b=32
if b in a:
    print("Element found")
else:
    print("Element not found")
```

13) List supports unpacking operation also.

```
a=[10,20,30]
b,c,d=a
print(a)
print(b)
print(c)
print(d)
```

14) Converting a string into a list also possible.

```
a="welcome"
b=list(a)
print(a)
```

15) List comprehensions:

The concept of generating elements into a list by writing some logic in a list is known as list comprehension.

Examples:

```
1) a=[i for i range(10)]  
    print(a)  
2) a=[i*i for i in range(10,20)]  
    print(a)
```

16) Functions:

- 1) len() function: It returns the length of the list.
- 2) max() function: It returns the maximum value in a list.
- 3) min() function: It returns the minimum value in a list.
- 4) sum() function: It returns sum of the elements in a list.
- 5) sorted() function: It returns sorted list.

Example:

```
a=[10,23,83,43,92,13]  
print(len(a))  
print(max(a))  
print(min(a))  
print(sum(a))  
print(sorted(a))
```

17) Methods:

- 1) append() method: It is used append an element at the end of the list.
- 2) count() method: It is used to count the no. of times appears a specified element in a list.
- 3) index() method: It is used to get the lowest index of specified element.
- 4) copy() method: It is used to create a copy of list.
- 5) remove() method: It is used to remove specified element.
- 6) extend() method: It is used to extend a list with another list.

7) sort() method: It is used to sort elements in a list.

8) reverse() method: It is used to reverse elements in a list.

9) clear() method: It is used to remove all elements from list.

10) pop() method: It is used to remove an element at the specified index.

11) insert() method: It is used to insert an element at the specified index.

Example:

```
a=[10,39,53,39,81,35]
```

```
print(a)
```

```
a.append(33)
```

```
print(a)
```

```
print(a.count(39))
```

```
print(a.index(81))
```

```
b=a.copy()
```

```
print(b)
```

```
a.remove(53)
```

```
print(a)
```

```
a.extend(b)
```

```
print(a)
```

```
a.sort()
```

```
print(a)
```

```
a.reverse()
```

```
print(a)
```

```
a.pop(2)
```

```
print(a)
```

```
a.insert(1,91)
print(a)
a.clear()
print(a)
```

Tuple:

- 1) Tuple can be created by using parenthesis.
- 2) Tuple can also be created by using tuple() function.
- 3) Tuple can also be created by assigning multiple values to a single variable.
- 4) Tuple can have homogeneous elements and heterogeneous elements.
- 5) Tuple allows duplicates.
- 6) Tuple supports both positive and negative indexing.
- 7) Tuple supports slice operations also.
- 8) Tuple is immutable.
- 9) Tuple does not support deleting an element.
- 10) Tuple can be deleted.
- 11) Tuple is an iterable object, elements of a Tuple can be iterated by using for loop or while loop.
- 12) Insertion order is preserved in a Tuple.
- 13) Tuple can be placed in another Tuple also. Tuple in a Tuple called as nested Tuple.
- 14) Tuple allows search operation also.
- 15) Tuple supports unpacking operation also.
- 16) Converting a string into a Tuple also possible.
- 17) Tuple comprehensions is not possible

18) Functions.

19) Tuple Methods.

1) count() => It is used to count the no. of times appears a specified element in a list.

2) index() => It is used to get the lowest index of specified element.

20) List in a tuple also possible

21) Tuple in a list also possible

Set:

1) Set can be created by using curly braces.

2) Set can also be created by using set() function.

3) Empty curly braces {} treated as dict.

4) Atleast one element if we write {10} then it will be treated as Set.

5) Set can have homogeneous elements and heterogeneous elements.

6) Set does not allow duplicates.

7) If we write duplicates, automatically ignores.

8) In a set insertion order is not preserved whereas in a list and tuple insertion order is preserved.

9) Set does not support indexing.

10) Set does not support slice operations.

11) Set is mutable because we can add an element by using add() method and we can remove an element by using remove() method.

12) Set can be deleted.

13) Set is an iterable object, elements of a Set can be iterated by using for loop or while loop.

14) Set allows search operation also.

- 15) Set supports unpacking operation also.
- 16) Converting a string into a Set also possible.
- 17) Set comprehensions is possible
- 18) Set in a set is not possible(Nested Set is not possible).
- 19) Tuple can be stored in a Set.
- 20) List cannot be stored in a Set.
- 21) Set can be stored in a List.
- 22) Set can be stored in a Tuple.
- 23) Set also supports mathematical operations like union(|), intersection(&), difference(-) & symmetric_difference(^).

Example:

```
a={10, 20, 30, 40}
b={30, 40, 50, 60}
c=a.union(b)
d=a.intersection(b)
e=a.difference(b)
f=a.symmetric_difference(b)
print(a)
print(b)
print(c)
print(d)
print(e)
print(f)
```

Frozenset:

- 1) Frozenset is a immutable version of a set.
- 2) Frozenset can be created by using frozenset() function.
- 3) We cannot add an element and we cannot remove an element because frozenset is immutable set.
- 4) Frozenset can be used as a key in a dictionary.

Dict:

- 1) Dictionary can be created by using curly braces.
- 2) Dictionary can also be created by using dict() function.
- 3) Dictionary maintains data as a key/value pairs.
- 4) Keys & values are separated with ":" symbol.
- 5) Here keys are immutable types only.

- 6) Here values can be any type

Note: int, float, complex, str, bool & tuple are immutable

- 7) Dictionary does not allow duplicate keys.
- 8) Values may be duplicated.
- 9) If we write duplicate keys, automatically ignores.
- 10) Insertion order is preserved.
- 11) Keys & values can be homogeneous or heterogeneous.
- 12) Dictionary does not support indexing.
- 13) Dictionary does not support slice operations.
- 14) Here values can be accessed by using keys.
- 15) Dictionary is mutable because new key & value pair can be added and existed value can be modified.

- 16) Dictionary is an iterable object. Key & Value pairs can be iterated by using for loop.
- 17) Dictionary comprehensions is possible
- 17) List cannot be used as a key
- 18) List can be used as a value
- 19) Tuple can be used as a key
- 20) Tuple can be used as a value
- 21) Set cannot be used as a key
- 22) Set can be used as a value
- 23) Dict supports deleting a key & value pair.
- 24) Dict can be deleted.
- 25) Dict allows search operation also.
- 26) Converting a string into a dict is not possible.
- 27) Dict supports unpacking operation also.

By

Mr. Venatesh Mansani

Naresh i Technologies